

Academic Project Report

Minimax Tic-Tac-Toe Artificial Intelligence

Course: CSE (AI & ML)

Submitted By: Prathmesh Tandon

Registration Number: 25BAI11352

Date: September 18, 2026

2. Introduction

The development of intelligent agents capable of making optimal decisions in competitive environments is a cornerstone of Artificial Intelligence. This project bridges theoretical game logic and practical software engineering by implementing a flawless automated opponent for the classic game of Tic-Tac-Toe. By utilizing recursive state evaluation, the program models how intelligent systems navigate constrained zero-sum environments.

3. Problem Statement

Traditional heuristic-based artificial intelligence often fails in edge cases because it relies on static rules rather than dynamic forecasting. The objective of this project is to develop a reliable decision-making agent that calculates optimal moves by exhaustively exploring all possible future game states, mathematically guaranteeing that the artificial intelligence will either win or draw, but never lose.

4. Functional Requirements

- **Environment Simulation:** The system must accurately simulate a 3x3 game board, allowing humans and the AI to claim distinct spaces via a console interface.
- **State Evaluation:** The program must continuously evaluate the board to detect win, loss, and draw states, assigning scores to terminal outcomes.
- **Decision Engine:** The system must implement the Minimax algorithm to recursively simulate human and AI turns, outputting the optimal coordinate for the AI's next move.
- **Input Validation:** The program must intercept invalid human inputs (e.g., out-of-bounds coordinates or attempting to overwrite an existing move) without crashing.

5. Non-Functional Requirements

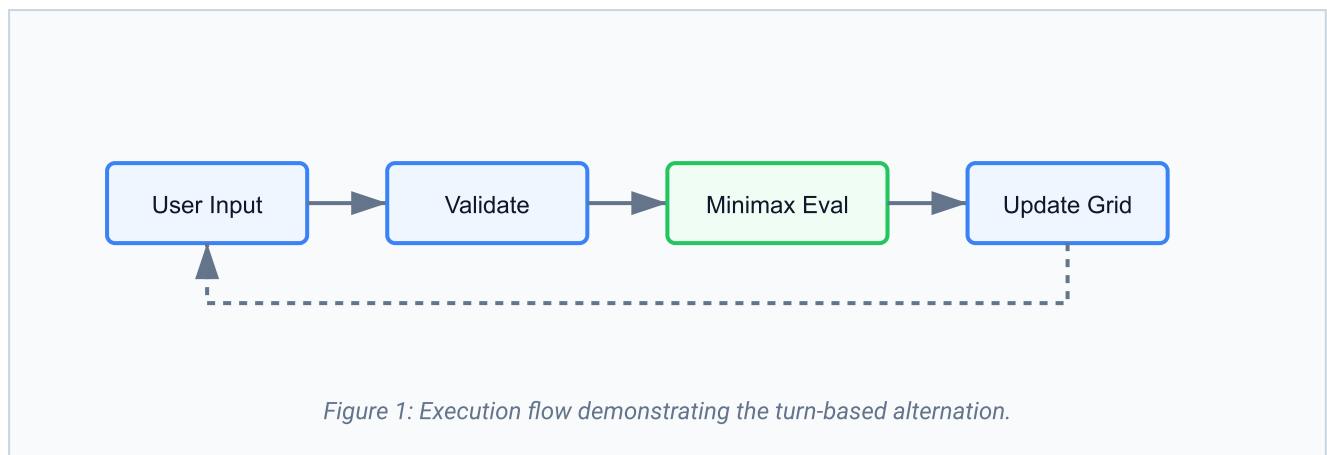
- **Performance:** The game tree evaluation must resolve and execute the AI's move in less than 500 milliseconds for a seamless user experience.
- **Modularity (Maintainability):** The codebase must be strictly separated into functional classes (Move, Board, AIPlayer, Game) rather than a monolithic script.
- **Reliability:** The AI must be completely unbeatable; it must achieve a 100% block rate against human forks and traps.
- **Usability:** The console interface must provide clear instructions, display the updated grid after every turn, and announce the final result cleanly.

6. System Architecture

The system utilizes a modular object-oriented architecture. The **Presentation Layer** is handled by standard I/O streams in the console. The **Logic Layer** is maintained by the ``Game`` and ``Board`` classes which handle state transitions. The **AI Layer** (``AIPlayer``) functions as an isolated computation engine that receives the current state, computes the game tree, and returns the mathematically best ``Move`` object.

7. Design Diagrams

Workflow Diagram (Core Game Loop)



Sequence Diagram

- 1. Setup:** `Main()` instantiates `Game()` and `Board()`.
- 2. Human Turn:** User enters coordinates -> `Game()` validates against `Board()`.
- 3. AI Turn:** `Game()` invokes `AIPlayer.findBestMove(Board)`.
- 4. Computation:** `AIPlayer` recursively clones/evaluates `Board` states to terminal depths.
- 5. Resolution:** `AIPlayer` returns `Move(row, col)` -> `Game()` applies `Move` -> Loop repeats.

Class Component Diagram

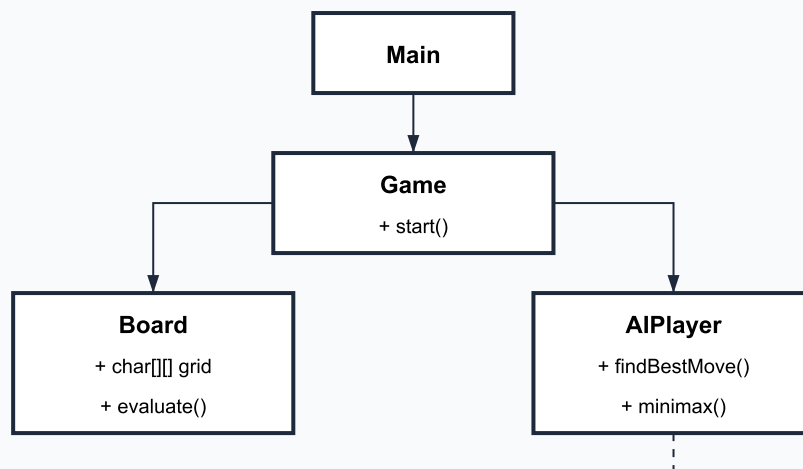


Figure 2: Component architecture and associations.

8. Design Decisions & Rationale

The Minimax algorithm was selected over simple rule-based heuristics. A heuristic approach requires manual programming of edge cases (e.g., "If center is open, take center; else if corner..."). Minimax, however, treats the game mathematically by mapping every potential outcome, dynamically assigning +10 for an AI win, -10 for a human win, and 0 for a draw. The AI simply acts to maximize this score. Additionally, Java was chosen for its strict static typing and object-oriented structure, which naturally models discrete entities like Boards and Moves.

9. Implementation Details

The game board is represented as a 2D character array (`char[][]`). The `evaluate()` function sweeps this matrix across all rows, columns, and diagonals to detect a terminal win state. The AI's decision process happens in `AIPlayer.java`, which iterates over every empty space on the board, temporarily places an 'X', calls `minimax()` to simulate the rest of the game, records the outcome score, removes the temporary 'X', and ultimately commits to the path yielding the highest score.

10. Screenshots / Results

Since this is a console application, the interface is completely text-based. I played through the game in my terminal. Below is a snapshot of the console output during a match where the AI successfully blocked my attempt to set up a double-win trap:

```
AI Minimax Tic-Tac-Toe. You are 'O', AI is 'X'.  
_ _ _  
_ X _  
_ _ O  
  
Enter row and column (0-2) separated by a space:  
2 0  
AI is making a move...  
_ _ X  
_ X _  
O _ O
```

Figure 3: Terminal output showing the AI blocking a player's win.

11. Testing

I tested the game manually by playing dozens of rounds in the terminal to see if I could break the game or trick the AI. My main focus areas were:

- **Handling Bad Inputs:** I tried typing out-of-bounds coordinates (like 5 5) and trying to pick squares that already had an X or O in them. The game correctly rejected the moves and asked me to try again without crashing.
- **Trying to Win:** I tried every Tic-Tac-Toe trick I knew to trap the AI. Every single time, the AI saw the trap coming and placed a block on the exact turn before I could finish it, which proved the Minimax algorithm was working exactly as intended.

12. Challenges Faced

The primary challenge was managing the state of the 2D array during deep recursion. Because Java passes object references by value, modifying the board inside a simulated Minimax iteration was permanently altering the actual game board. This was resolved by implementing a rigorous backtracking structure: explicitly resetting `board.grid[i][j] = '_'` immediately after the recursive call resolved, ensuring the board returned to its exact previous state before the next branch was explored.

13. Learnings & Key Takeaways

This project provided profound insights into the application of theoretical computer science concepts. Key takeaways include:

- **Game Theory in Code:** Translating abstract concepts like zero-sum games and perfect information into executable loops and arrays.
- **Recursion and Backtracking:** Gaining a strong practical understanding of how call stacks handle recursive depth, and the absolute necessity of state cleanup (backtracking) when traversing complex node trees.
- **Separation of Concerns:** Discovering how modularizing the AI completely separate from the `Game` interface makes debugging algorithmic logic significantly easier than a monolithic file.

14. Future Enhancements

While the current implementation is mathematically complete for Tic-Tac-Toe, future expansions could include:

- **Alpha-Beta Pruning:** Optimizing the AI by terminating branch evaluations early if a demonstrably worse outcome is detected, which is necessary for scaling to larger games like Connect 4 or Chess.
- **Graphical User Interface (GUI):** Migrating from a console-based application to a Java Swing or JavaFX interactive desktop window.
- **Dynamic Board Sizing:** Allowing the board to scale to 4x4 or 5x5 variations to increase computational complexity.

15. References

- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach (4th ed.)*. Pearson. (Chapter on Adversarial Search and Game Playing).
- Oracle Java SE Documentation. (2023). Collections and Arrays frameworks.
- VITyarthi Subject Syllabus: Fundamentals of AI and ML Concepts.